s2_g8_net_Lec10_ChatPDF_S_CoT_AU2440276_Mant

L10_S2.pdf
PDF

You are generating a lecture scribe intended as exam reference material for a closed-notes / reading-based exam.

Authoritative Context (Strict)
The only information you may use is the material explicitly provided as source content for the lecture.
This may include lecture slides, lecture PDFs, textbook excerpts, handwritten notes, or any combination thereof.

The lecture slides or primary lecture document define the structure, ordering, scope, and core content of the lecture and must be fully and faithfully represented in the lecture scribe.

Any supplementary material (such as handwritten notes or textbook excerpts) is to be treated as authoritative lecture material intended to:
-> clarify core content
-> add intermediate steps omitted in primary material
-> state assumptions or conditions mentioned during lecture
-> annotate or expand directly corresponding lecture material

If supplementary material contains:
-> clarifications,
-> intermediate steps,
-> assumptions,
-> annotations,
-> or details not explicitly written in the primary lecture document but directly corresponding to it,

they must be integrated at the appropriate location aligned with the primary lecture structure.

Supplementary material must not replace, overshadow, or reorder content that appears in the primary lecture document.

You must not use outside knowledge.
You must not invent definitions, notation, results, proofs, derivations, examples, or assumptions.

Prompting Style: Structured Reasoning (with Chain-of-Thought)

Use careful, step-by-step reasoning internally to construct the lecture scribe.

The lecture scribe must be produced in a single, complete, logically consistent output that strictly follows the provided source material.

Objective

Produce a faithful, structured reconstruction of what was taught in the lecture, such that a student who studies only this document can reliably revise the material for the exam.

The scribe must allow the student to:
-> Recall definitions and notation
-> Reconstruct proofs and derivations
-> Follow worked examples step by step
-> Understand logical dependencies between concepts

Content Requirements (Include Where Applicable)

-> All definitions and notation exactly as introduced
-> All assumptions, conditions, and domains of validity
-> Statements of theorems, propositions, lemmas, or results
-> Proofs or proof sketches reconstructed step by step as presented
-> Worked examples shown in full, with all intermediate steps
-> Explicit logical dependencies between concepts

All primary lecture content must be explicitly represented in the scribe.
Supplementary material may add detail but must not introduce independent structure or unrelated content.

Reasoning Requirement

When a proof, derivation, or example appears in the provided material:
-> Reproduce the explicit logical steps required to follow it
-> Do not add intuition, motivation, alternative arguments, shortcuts, or commentary not present in the provided material

Structure and Style Constraints

-> Follow the same order and scope as the primary lecture document
-> Use precise, formal, exam-appropriate language
-> Be concise but complete; do not omit steps shown in the provided material
-> Do not simplify, re-teach, or editorialize
-> Do not summarize at a high level unless explicitly done in the lecture material

Prohibitions

-> No invented content
-> No external examples
-> No intuition or explanations beyond the provided material
-> No references to prompting, instructions, or the existence of an AI

Final Output

A clean, logically organized lecture scribe suitable for direct exam preparation.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Dhaval Patel, PhD
February 5, 2026

Source: Lecture Slides (L10_S2.pdf)
L10_S2

---

1. Outline of the Lecture

1. Min-Cut Problem
    - Why use min-cut?
    - What is min-cut?
    - Successful min-cut run
    - Unsuccessful min-cut run
    - Max-Flow Min-Cut Theorem
2. Deterministic Min-Cut Algorithm
    - Stoer–Wagner minimum cut algorithm
    - Pseudocode
3. Randomized Min-Cut Algorithm
    - Why randomized algorithm?
    - Karger's Randomized Algorithm
    - Pseudocode
    - Comparison: Deterministic vs Randomized
    - Theorem for min-cut set
    - Python Simulation

---

2. Min-Cut Problem

2.1 Why Use Min-Cut?

The min-cut algorithm is used in applications related to:

- **Network connectivity**
- **Reliability**
- **Optimization**

Applications

1. **Network Design**
    - Helps improve communication efficiency.

- Used to find the minimum capacity cut.
- Optimizes network flow.

2. **Communication Networks**
   - Helps understand vulnerability to failures.
   - Assists in building robust and fault-tolerant networks.
3. **VLSI Design**
   - Used for partitioning circuits into smaller components.
   - Reduces interconnectivity complexity.

(Reference in slides: Section 1.5 – Application: A Randomized Min-Cut Algorithm in *Probability and Computing (2nd Edition)*.)
L10_S2

2.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices,

The **minimum cut (min-cut)** problem is:

> To find a minimum cardinality cut-set in G.

2.3 Edge Contraction (Core Operation)

The main operation in the algorithm is **edge contraction**.

Definition: Edge Contraction

Edge contraction is:

> An operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

When contracting edge (u, v):

1. Merge vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph:
    - May have **parallel edges**
    - Has **no self-loops**

(The diagrams on pages 15–16 illustrate the contraction process step-by-step.) L10_S2

---

3. Successful and Unsuccessful Min-Cut Runs

3.1 Successful Min-Cut Run

A **successful min-cut run**:

> Refers to success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure on page 19 shows a sequence of contractions that results in the correct minimum cut. L10_S2

3.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run**:

> Refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The figure on page 22 shows a contraction sequence that eliminates critical edges early and fails to return the true minimum cut. L10_S2

---

4. Max-Flow Min-Cut Theorem

Statement

> In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions

Capacity of a Cut

$$\text{Capacity of a cut} = \sum \text{capacities of edges oriented from } X \text{ to } Y$$

where:

- X contains the source,
- Y contains the sink.

Minimum Cut

- The cut with the **smallest possible capacity**.
- **Minimum cut capacity** = capacity of the minimum cut.

Maximum Flow

- The largest possible flow from source S to sink T.

(The diagram on page 26 visually illustrates the equality between maximum flow and minimum cut capacity.)

L10_S2

---

5. Deterministic Min-Cut Algorithm

Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of graph G.

Let $G/\{s,t\}$ denote the graph obtained by merging s and t.

Theorem

A minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$.

Justification

Either:

- There exists a minimum cut separating s and t,
  $\rightarrow$ Then a minimum s-t-cut is the minimum cut.

Or:

- No minimum cut separates s and t,
  $\rightarrow$ Then a minimum cut of $G/\{s,t\}$ gives the answer.

L10_S2

Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
pgsql

A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```
pgsql

while |V| ≥ 1 do
    choose any a from V;
    MinimumCutPhase(G, a);
    if cut-of-the-phase is lighter than current minimum cut then
        store cut-of-the-phase as current minimum cut;
    shrink G by merging the two vertices added last;
return the minimum cut;
```

L10_S2

Time Complexity

Stoer–Wagner algorithm time complexity:

$$O(V \cdot E + V^2 \log V)$$

where:

- V = number of vertices
- E = number of edges

L10_S2

---

6. Randomized Min-Cut Algorithm

6.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Advantages Listed in Slides

- Efficiency
- Parallelization

- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

L10_S2

6.2 Karger's Randomized Algorithm

The algorithm repeatedly:

1. Picks a random edge.
2. Contracts it.
3. Repeats until only two vertices remain.
4. The remaining edges between the two vertices form a cut.

The diagram on page 34 shows:

- Random edge selection,
- Fusion of vertices,
- An example where the output cut is not minimal.

L10_S2

6.3 Recursive Randomized Min-Cut Pseudocode

Algorithm 3: RECURSIVE-RANDOMIZED-MIN-CUT(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

```
vbnet

if n ≤ α^3 then
    C ← min-cut of G using brute force (exhaustive search);
else
    for i ← 1 to α do
        G' ← multigraph obtained by applying
                n − ⌈n/√α⌉ random contraction steps on G;
        C' ← RECURSIVE-RANDOMIZED-MIN-CUT(G', α);
        if i = 1 or |C'| < |C| then
```

```
            C ← C';
return C;
```

L10_S2

6.4 Time Complexity

Karger's randomized algorithm:

$$O(V^2)$$

where V is the number of vertices.

L10_S2

---

7. Comparison: Deterministic vs Randomized

Deterministic Min-Cut

- Guarantees exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer–Wagner: $O(V \cdot E + V^2 \log V)$

Randomized Min-Cut

- Provides approximate minimum cut with high probability.
- Karger's algorithm: $O(V^2)$

The choice depends on the specific problem.

L10_S2

---

8. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

L10_S2

---

9. Python Simulation

Instructions from slides:

- Open Campuswire post for Lecture 10.
- Download the `.ipynb` file posted in comments.

L10_S2

End of Lecture 10 Scribe

This document faithfully reconstructs Lecture 10 content from the provided slides.

Sources

L10_S2.pdf
PDF

You are generating a lecture scribe intended as exam reference material for a closed-notes / reading-based exam.

Authoritative Context (Strict)
The only information you may use is the material explicitly provided as source content for the lecture.
This may include lecture slides, lecture PDFs, textbook excerpts, handwritten notes, or any combination thereof.

The lecture slides or primary lecture document define the structure, ordering, scope, and core content of the lecture and must be fully and faithfully represented in the lecture scribe.

Any supplementary material (such as handwritten notes or textbook excerpts) is to be treated as authoritative lecture material intended to:
-> clarify core content
-> add intermediate steps omitted in primary material
-> state assumptions or conditions mentioned during lecture
-> annotate or expand directly corresponding lecture material

If supplementary material contains:
-> clarifications,
-> intermediate steps,
-> assumptions,
-> annotations,
-> or details not explicitly written in the primary lecture document but directly corresponding to it,

they must be integrated at the appropriate location aligned with the primary lecture structure.

Supplementary material must not replace, overshadow, or reorder content that appears in the primary lecture document.

You must not use outside knowledge.
You must not invent definitions, notation, results, proofs, derivations, examples, or assumptions.

Prompting Style: Structured Reasoning (with Chain-of-Thought)

Use careful, step-by-step reasoning internally to construct the lecture scribe.

The lecture scribe must be produced in a single, complete, logically consistent output that strictly follows the provided source material.

Objective

Produce a faithful, structured reconstruction of what was taught in the lecture, such that a student who studies only this document can reliably revise the material for the exam.

The scribe must allow the student to:
-> Recall definitions and notation
-> Provide formulas which are useful for exam prep

-> Reconstruct proofs and derivations
-> Follow worked examples step by step
-> Understand logical dependencies between concepts

Content Requirements (Include Where Applicable)

-> All definitions and notation exactly as introduced
-> All assumptions, conditions, and domains of validity
-> Statements of theorems, propositions, lemmas, or results
-> Proofs or proof sketches reconstructed step by step as presented
-> Worked examples shown in full, with all intermediate steps
-> Explicit logical dependencies between concepts

All primary lecture content must be explicitly represented in the scribe.
Supplementary material may add detail but must not introduce independent structure or unrelated content.

Reasoning Requirement

When a proof, derivation, or example appears in the provided material:
-> Reproduce the explicit logical steps required to follow it
-> Do not add intuition, motivation, alternative arguments, shortcuts, or commentary not present in the provided material

Structure and Style Constraints

-> Follow the same order and scope as the primary lecture document
-> Use precise, formal, exam-appropriate language
-> Be concise but complete; do not omit steps shown in the provided material
-> Do not simplify, re-teach, or editorialize
-> Do not summarize at a high level unless explicitly done in the lecture material

Prohibitions

-> No invented content
-> No external examples
-> No intuition or explanations beyond the provided material
-> No references to prompting, instructions, or the existence of an AI

Final Output

A clean, logically organized lecture scribe suitable for direct exam preparation.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Dhaval Patel, PhD
February 5, 2026

Source: Lecture Slides (L10_S2.pdf)
L10_S2

---

1. Outline

- Min-Cut Problem
  - Why use min-cut?
  - What is min-cut?
  - Successful min-cut run
  - Unsuccessful min-cut run
  - Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm
  - Stoer–Wagner minimum cut algorithm
  - Pseudocode
- Randomized Min-Cut Algorithm
  - Why randomized algorithm?
  - Karger's Randomized Algorithm
  - Pseudocode
  - Comparison: Deterministic vs Randomized
  - Theorem for min-cut set
  - Python Simulation
  - Class Participation

---

2. Min-Cut Problem

2.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to:

- Network connectivity

- Reliability
- Optimization

Applications

1. **Network Design**
    - Improves efficiency of communication.
    - Optimizes network flow.
    - Used to find the minimum capacity cut.
2. **Communication Networks**
    - Helps understand vulnerability of networks to failures.
    - Used to build robust and fault-tolerant communication networks.
3. **VLSI Design**
    - Useful for partitioning circuits into smaller components.
    - Leads to reduced interconnectivity complexity.

Reference: Section 1.5 – *Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition)*.

L10_S2

2.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices,

The **minimum cut (min-cut)** problem is:

> To find a minimum cardinality cut-set in G.

The slides state that min-cut algorithms such as Karger's algorithm are random and can be sensitive to the initial choice of edges. If critical edges are contracted early, the algorithm may return a smaller cut.

L10_S2

2.3 Edge Contraction

The main operation in the algorithm is **edge contraction**.

Definition

Edge contraction is:

> An operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

When contracting edge (u, v):

1. Merge vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph:
   - May have parallel edges.
   - Has no self-loops.

The diagrams on pages 15–16 illustrate the contraction process step-by-step, showing vertex merging and elimination of self-loops.
L10_S2

---

3. Successful and Unsuccessful Min-Cut Runs

3.1 Successful Min-Cut Run

A **successful min-cut run**:

> Refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure on page 19 shows a contraction sequence resulting in the correct minimum cut.
L10_S2

3.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run**:

> Refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The figure on page 22 shows a contraction sequence where critical edges are contracted early, leading to failure.
L10_S2

---

4. Max-Flow Min-Cut Theorem

Statement

> In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions

Capacity of a Cut

$$\text{Capacity of a cut} = \sum \text{capacities of edges in the cut oriented from } X \text{ to } Y$$

Minimum Cut

- The cut in the network that has the smallest possible capacity.
- **Minimum cut capacity** = capacity of the minimum cut.

Maximum Flow

- The largest possible flow from source S to sink T.

The diagram on page 26 illustrates the equality between maximum flow and minimum cut capacity.

L10_S2

---

5. Deterministic Min-Cut Algorithm

Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G.

Let $G/\{s,t\}$ denote the graph obtained by merging s and t.

Theorem

A minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$.

Justification (as stated)

- If there exists a minimum cut of G that separates s and t, then a minimum s-t-cut of G is a minimum cut of G.
- Otherwise, a minimum cut of $G/\{s,t\}$ does the job.

L10_S2

Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
pgsql

A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```
pgsql

while |V| ≥ 1 do
    choose any a from V;
    MinimumCutPhase(G, a);
    if the cut-of-the-phase is lighter than the current minimum cut then
        store the cut-of-the-phase as the current minimum cut;
    shrink G by merging the two vertices added last;
return the minimum cut;
```

L10_S2

Time Complexity (Deterministic)

Stoer–Wagner algorithm:

$$O(V \cdot E + V^2 \log V)$$

where:

- V = number of vertices
- E = number of edges

L10_S2

---

6. Randomized Min-Cut Algorithm

6.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Advantages listed:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

L10_S2

6.2 Karger's Randomized Algorithm

The figure on page 34 shows:

- Random selection of edge b, removal, and fusion of two vertices.
- Subsequent selection of edge d, removal, and fusion.
- An example where the output cut is not minimal.

The slide states:

> The output cut produced here is {a, c, e}, but the minimal cut is either {b, e} or {a, d}.

L10_S2

6.3 Recursive Randomized Min-Cut (Pseudocode)

Algorithm 3: RECURSIVE-RANDOMIZED-MIN-CUT(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

```
vbnet

if n ≤ α^3 then
    C ← a min-cut of G found using brute force (exhaustive search);
else
    for i ← 1 to α do
        G' ← multigraph obtained by applying
              n − ⌈n/√α⌉ random contraction steps on G;
        C' ← RECURSIVE-RANDOMIZED-MIN-CUT(G', α);
        if i = 1 or |C'| < |C| then
            C ← C';
return C;
```

L10_S2

Time Complexity (Randomized)

Karger's algorithm:

$$O(V^2)$$

where V is the number of vertices.

L10_S2

7. Comparison: Deterministic vs Randomized

The specific problem determines the approach.

Deterministic Min Cut

- Always guarantees exact minimum cut.
- May have higher time complexity for large graphs.
- Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min Cut

- Provides approximate minimum cut with high probability.
- Time complexity:

$$O(V^2)$$

L10_S2

8. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

L10_S2

9. Python Simulation

Instructions:

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments.

L10_S2

End of Lecture 10

Sources

L10_S2.pdf
PDF

You are generating a lecture scribe intended as exam reference material for a closed-notes / reading-based exam.

Authoritative Context (Strict)
The only information you may use is the material explicitly provided as source content for the lecture.
This may include lecture slides, lecture PDFs, textbook excerpts, handwritten notes, or any combination thereof.

The lecture slides or primary lecture document define the structure, ordering, scope, and core content of the lecture and must be fully and faithfully represented in the lecture scribe.

Any supplementary material (such as handwritten notes or textbook excerpts) is to be treated as authoritative lecture material intended to:
-> clarify core content
-> add intermediate steps omitted in primary material
-> state assumptions or conditions mentioned during lecture
-> annotate or expand directly corresponding lecture material

If supplementary material contains:
-> clarifications,
-> intermediate steps,
-> assumptions,
-> annotations,
-> or details not explicitly written in the primary lecture document but directly corresponding to it,

they must be integrated at the appropriate location aligned with the primary lecture structure.

Supplementary material must not replace, overshadow, or reorder content that appears in the primary lecture document.

You must not use outside knowledge.
You must not invent definitions, notation, results, proofs, derivations, examples, or assumptions.

Prompting Style: Structured Reasoning (with Chain-of-Thought)

Use careful, step-by-step reasoning internally to construct the lecture scribe.

The lecture scribe must be produced in a single, complete, logically consistent output that strictly follows the provided source material.

Objective

Produce a faithful, structured reconstruction of what was taught in the lecture, such that a student who studies only this document can reliably revise the material for the exam.

The scribe must allow the student to:
-> Recall definitions and notation
-> Provide formulas which are useful for exam prep

-> Reconstruct proofs and derivations
-> Follow worked examples step by step
-> Understand logical dependencies between concepts

Content Requirements (Include Where Applicable)

-> All definitions and notation exactly as introduced
-> All assumptions, conditions, and domains of validity
-> Statements of theorems, propositions, lemmas, or results
-> Proofs or proof sketches reconstructed step by step as presented
-> Worked examples shown in full, with all intermediate steps
-> Explicit logical dependencies between concepts

All primary lecture content must be explicitly represented in the scribe.
Supplementary material may add detail but must not introduce independent structure or unrelated content.

Reasoning Requirement

When a proof, derivation, or example appears in the provided material:
-> Reproduce the explicit logical steps required to follow it
-> Do not add intuition, motivation, alternative arguments, shortcuts, or commentary not present in the provided material

Structure and Style Constraints

-> Follow the same order and scope as the primary lecture document
-> Use precise, formal, exam-appropriate language
-> Be concise but complete; do not omit steps shown in the provided material
-> Do not simplify, re-teach, or editorialize
-> Do not summarize at a high level unless explicitly done in the lecture material

Prohibitions

-> No invented content
-> No external examples
-> No intuition or explanations beyond the provided material

-> No references to prompting, instructions, or the existence of an AI

Final Output

A clean, logically organized lecture scribe suitable for direct exam preparation.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Dhaval Patel, PhD
February 5, 2026

Source: Lecture Slides (L10_S2.pdf)

L10_S2

---

1. Outline

1. Min-Cut Problem
    - Why use min-cut?
    - What is min-cut?
    - Successful min-cut run
    - Unsuccessful min-cut run
    - Max-Flow Min-Cut Theorem
2. Deterministic Min-Cut Algorithm
    - Stoer–Wagner minimum cut algorithm
    - Pseudocode
3. Randomized Min-Cut Algorithm
    - Why randomized algorithm?
    - Karger's Randomized Algorithm
    - Pseudocode
    - Comparison: Deterministic Min-Cut vs Randomized Min-Cut Algorithm
    - Theorem for min-cut set
    - Python Simulation
    - Class Participation

---

2. Min-Cut Problem

2.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to:

- Network connectivity
- Reliability
- Optimization

Applications (as listed in slides)

1. **Network Design**
    - Min cut helps in improving the efficiency of communication.
    - Optimizes network flow.
    - Used to find the minimum capacity cut.
2. **Communication Networks**
    - Useful for understanding vulnerability of networks to failures.
    - Helps in building robust and fault-tolerant communication networks.
3. **VLSI Design**
    - Useful for partitioning circuits into smaller components.
    - Leads to reduced interconnectivity complexity.

Reference given in slides:
Section 1.5 – *Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition)*.
L10_S2

2.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices,

The **minimum cut (min-cut)** problem is:

> To find a minimum cardinality cut-set in G.

Slides additionally state:

- Min-cut algorithms such as Karger's algorithm are random.

- They can be sensitive to the initial choice of edges.
- If critical edges are contracted early, the algorithm might find a smaller cut.

L10_S2

2.3 Edge Contraction

The main operation in the algorithm is **edge contraction**.

Definition

Edge contraction is:

> An operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

When contracting edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph:
    - May have parallel edges
    - Has no self-loops

The diagrams on pages 15–16 show:

- Original graph
- Selection of an edge
- Merging of two vertices
- Resulting graph with parallel edges but no self-loops

L10_S2

---

3. Successful and Unsuccessful Min-Cut Runs

3.1 Successful Min-Cut Run

A **successful min-cut run**:

> Refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure on page 19 illustrates:

- A sequence of contractions
- Final partition producing the correct minimum cut

L10_S2

3.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run**:

> Refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The figure on page 22 illustrates:

- A sequence of contractions
- Early contraction of critical edges
- Final partition that is not the true minimum cut

L10_S2

---

4. Max-Flow Min-Cut Theorem

Statement

> In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions Introduced in Slides

Capacity of a Cut

$$\text{Capacity of a cut} = \sum \text{capacity of edges in the cut oriented from a vertex } \in X \text{ to a vertex } \in Y$$

Minimum Cut

- The cut in the network that has the smallest possible capacity.
- **Minimum cut capacity** = capacity of the minimum cut.

Maximum Flow

- The largest possible flow from source S to sink T.

The diagram on page 26 illustrates:

- Partition of vertices into two sets
- Flow values on edges
- Equality between maximum flow and minimum cut capacity

L10_S2

---

5. Deterministic Min-Cut Algorithm

5.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G.

Let $G/\{s,t\}$ denote the graph obtained by merging s and t.

Theorem (as stated in slides)

A minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$.

Justification

The theorem holds since:

- Either there is a minimum cut of G that separates s and t,
  $\rightarrow$ then a minimum s-t-cut of G is a minimum cut of G;
- Or there is none,
  $\rightarrow$ then a minimum cut of $G/\{s,t\}$ does the job.

L10_S2

5.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
pgsql


A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```pgsql
while |V| ≥ 1 do
    choose any a from V;
    MinimumCutPhase(G, a);
    if the cut-of-the-phase is lighter than the current minimum cut then
        store the cut-of-the-phase as the current minimum cut;
    shrink G by merging the two vertices added last;
return the minimum cut;
```

L10_S2

5.3 Time Complexity (Deterministic)

Stoer–Wagner algorithm:

$$O(V \cdot E + V^2 \log V)$$

where:

- V = number of vertices
- E = number of edges

L10_S2

---

6. Randomized Min-Cut Algorithm

6.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Additional properties listed:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

L10_S2

6.2 Karger's Randomized Algorithm

The slide on page 34 illustrates:

1. Pick edge b, remove it and fuse its two corners into one.
2. Pick edge d, remove it and fuse its two corners into one.

The slide text states:

> The output cut produced here is {a, c, e}, but the minimal cut is either {b, e} or {a, d}.

This demonstrates that random edge selection may produce a non-minimal cut.

L10_S2

6.3 Pseudocode

Algorithm 3: RECURSIVE-RANDOMIZED-MIN-CUT(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

```
vbnet

if n ≤ α^3 then
    C ← a min-cut of G found using brute force (exhaustive search);
else
    for i ← 1 to α do
        G' ← multigraph obtained by applying
                n − ⌈n/√α⌉ random contraction steps on G;
        C' ← RECURSIVE-RANDOMIZED-MIN-CUT(G', α);
        if i = 1 or |C'| < |C| then
            C ← C';
return C;
```

L10_S2

6.4 Time Complexity (Randomized)

Karger's algorithm:

$$O(V^2)$$

where V is the number of vertices.

L10_S2

---

7. Comparison: Deterministic vs Randomized

The slides state:

> Any specific problem is responsible for deciding the approach.

Deterministic Min Cut

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min Cut

- Provides an approximate minimum cut with high probability.
- Time complexity:

$$O(V^2)$$

L10_S2

---

8. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

L10_S2

---

9. Python Simulation

Instructions provided:

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments.

L10_S2

End of Lecture 10

Sources

L10_S2.pdf
PDF

You are generating a lecture scribe intended as exam reference material for a closed-notes / reading-based exam.

Authoritative Context (Strict)
The only information you may use is the material explicitly provided as source content for the lecture.
This may include lecture slides, lecture PDFs, textbook excerpts, handwritten notes, or any combination thereof.

The lecture slides or primary lecture document define the structure, ordering, scope, and core content of the lecture and must be fully and faithfully represented in the lecture scribe.

Any supplementary material (such as handwritten notes or textbook excerpts) is to be treated as authoritative lecture material intended to:
-> clarify core content
-> add intermediate steps omitted in primary material
-> state assumptions or conditions mentioned during lecture
-> annotate or expand directly corresponding lecture material

If supplementary material contains:
-> clarifications,
-> intermediate steps,
-> assumptions,
-> annotations,
-> or details not explicitly written in the primary lecture document but directly corresponding to it,

they must be integrated at the appropriate location aligned with the primary lecture structure.

Supplementary material must not replace, overshadow, or reorder content that appears in the primary lecture document.

You must not use outside knowledge.
You must not invent definitions, notation, results, proofs, derivations, examples, or assumptions.

Prompting Style: Structured Reasoning (with Chain-of-Thought)

Use careful, step-by-step reasoning internally to construct the lecture scribe.

The lecture scribe must be produced in a single, complete, logically consistent output that strictly follows the provided source material.

Objective

Produce a faithful, structured reconstruction of what was taught in the lecture, such that a student who studies only this document can reliably revise the material for the exam.

The scribe must allow the student to:
-> Recall definitions and notation
-> Provide formulas which are useful for exam prep

-> Reconstruct proofs and derivations
-> Follow worked examples step by step
-> Understand logical dependencies between concepts

Content Requirements (Include Where Applicable)

-> All definitions and notation exactly as introduced
-> All assumptions, conditions, and domains of validity
-> Statements of theorems, propositions, lemmas, or results
-> Proofs or proof sketches reconstructed step by step as presented
-> Worked examples shown in full, with all intermediate steps
-> Explicit logical dependencies between concepts

All primary lecture content must be explicitly represented in the scribe.
Supplementary material may add detail but must not introduce independent structure or unrelated content.

Reasoning Requirement

When a proof, derivation, or example appears in the provided material:
-> Reproduce the explicit logical steps required to follow it
-> Do not add intuition, motivation, alternative arguments, shortcuts, or commentary not present in the provided material

Structure and Style Constraints

-> Follow the same order and scope as the primary lecture document
-> Use precise, formal, exam-appropriate language
-> Be concise but complete; do not omit steps shown in the provided material
-> Do not simplify, re-teach, or editorialize
-> Do not summarize at a high level unless explicitly done in the lecture material

Prohibitions

-> No invented content
-> No external examples
-> No intuition or explanations beyond the provided material
-> No references to prompting, instructions, or the existence of an AI

Final Output

A clean, logically organized lecture scribe suitable for direct exam preparation.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Dhaval Patel, PhD
February 5, 2026

Source: Lecture Slides (L10_S2.pdf)
L10_S2

---

1. Outline (as given in slides)

- Min-Cut Problem
  - Why use min-cut?
  - What is min-cut?
  - Successful min-cut run
  - Unsuccessful min-cut run
  - Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm
  - Stoer–Wagner minimum cut algorithm
  - Pseudocode
- Randomized Min-Cut Algorithm
  - Why randomized algorithm?
  - Karger's Randomized Algorithm
  - Pseudocode
  - Comparison: Deterministic Min-Cut vs Randomized Min-Cut Algorithm
  - Theorem for min-cut set
  - Python Simulation
  - Class Participation

---

2. Min-Cut Problem

2.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to:

- Network connectivity
- Reliability
- Optimization

Applications (explicitly listed)

1. **Network Design**

- Min cut helps in improving the efficiency of communication.
- Optimizes network flow.
- Used to find the minimum capacity cut.

2. **Communication Networks**
   - Useful for understanding the vulnerability of networks to failures.
   - Helps in building robust and fault-tolerant communication networks.
3. **VLSI Design**
   - Useful for partitioning circuits into smaller components.
   - Leads to reduced interconnectivity complexity.

Reference mentioned in slides:
Section 1.5 – *Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition)*.

L10_S2

2.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices,

The **minimum cut (min-cut)** problem is:

> To find a minimum cardinality cut-set in G.

Slides additionally state:

- Min-cut algorithms such as Karger's algorithm are random.
- They can be sensitive to the initial choice of edges.
- If the algorithm contracts critical edges early, it might find a smaller cut.

L10_S2

2.3 Edge Contraction

The main operation in the algorithm is **edge contraction**.

Definition

Edge contraction is:

> An operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

When contracting edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph:
    - May have parallel edges
    - Has no self-loops

The diagrams on pages 15–16 illustrate:

- Selection of an edge
- Merging of its endpoints
- Elimination of self-loops
- Possible creation of parallel edges

L10_S2

---

3. Successful and Unsuccessful Min-Cut Runs

3.1 Successful Min-Cut Run

A **successful min-cut run**:

> Refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure on page 19 shows:

- A sequence of contractions
- Final partition that corresponds to the minimum cut

L10_S2

3.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run**:

> Refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The figure on page 22 shows:

- Contraction of critical edges
- Final partition that is not the true minimum cut

L10_S2

---

4. Max-Flow Min-Cut Theorem

Statement

> In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions (as introduced)

Capacity of a Cut

$$\text{Capacity of a cut} = \sum \text{capacity of edges in the cut oriented from a vertex } \in X \text{ to a vertex } \in Y$$

Minimum Cut

- The cut in the network that has the smallest possible capacity.
- **Minimum cut capacity** = the capacity of the minimum cut.

Maximum Flow

- The largest possible flow from source S to sink T.

The diagram on page 26 visually demonstrates:

- Partition of vertices
- Flow values on edges
- Equality between maximum flow and minimum cut capacity

L10_S2

---

5. Deterministic Min-Cut Algorithm

5.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G.

Let $G/\{s,t\}$ be the graph obtained by merging s and t.

Theorem (exact wording from slides)

A minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$.

Justification

The theorem holds since:

- Either there is a minimum cut of G that separates s and t,
  $\rightarrow$ then a minimum s-t-cut of G is a minimum cut of G;
- Or there is none,
  $\rightarrow$ then a minimum cut of $G/\{s,t\}$ does the job.

L10_S2

5.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
pgsql

A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```
pgsql

while |V| ≥ 1 do
    choose any a from V;
    MinimumCutPhase(G, a);
    if the cut-of-the-phase is lighter than the current minimum cut then
        store the cut-of-the-phase as the current minimum cut;
    shrink G by merging the two vertices added last;
return the minimum cut;
```

L10_S2

5.3 Time Complexity (Deterministic)

Stoer–Wagner algorithm:

$$O(V \cdot E + V^2 \log V)$$

where:

- V = number of vertices
- E = number of edges

L10_S2

---

6. Randomized Min-Cut Algorithm

6.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Additional characteristics listed:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

L10_S2

6.2 Karger's Randomized Algorithm

The diagram on page 34 shows:

1. Pick edge b, remove it and fuse its two corners into one.
2. Pick edge d, remove it and fuse its two corners into one.

The slide explicitly states:

> The output cut produced here is {a, c, e}, but the minimal cut is either {b, e} or {a, d}.

This example illustrates that random contraction can yield a non-minimal cut.

L10_S2

6.3 Pseudocode

Algorithm 3: RECURSIVE-RANDOMIZED-MIN-CUT(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

```
vbnet

if n ≤ α^3 then
    C ← a min-cut of G found using brute force (exhaustive search);
else
    for i ← 1 to α do
        G' ← multigraph obtained by applying
              n − ⌈n/√α⌉ random contraction steps on G;
        C' ← RECURSIVE-RANDOMIZED-MIN-CUT(G', α);
        if i = 1 or |C'| < |C| then
            C ← C';
return C;
```

L10_S2

6.4 Time Complexity (Randomized)

Karger's algorithm:

$$O(V^2)$$

where V is the number of vertices.

L10_S2

7. Comparison: Deterministic vs Randomized

The slides state:

> Any specific problem is responsible for deciding the approach.

Deterministic Min Cut

- Always guarantees an exact minimum cut.

- May have higher time complexity for large graphs.
- Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min Cut

- Provides an approximate minimum cut with high probability.
- Time complexity:

$$O(V^2)$$

L10_S2

---

8. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

This is the probability guarantee stated in the slides.

L10_S2

---

9. Python Simulation

Instructions given:

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments.

L10_S2

End of Lecture 10 Scribe

Sources